

Objectives and Functions of Modern Operating Systems

Operating System Principles -
SKU 6.1.1

Introduction

- ▶ Modern operating systems (OS) are essential software that manage computer hardware and software resources.
- ▶ They provide an environment where users and applications can run efficiently.

Key Objectives of Modern Operating Systems

- ▶ • Convenience: Make the computer system easier to use
- ▶ • Efficiency: Ensure optimal use of hardware resources
- ▶ • Reliability: Provide stable and error-free operation
- ▶ • Security: Protect data and resources from unauthorized access
- ▶ • Scalability: Support growing workloads and users

Core Functions of an Operating System

- ▶ • Process Management
- ▶ • Memory Management
- ▶ • File System Management
- ▶ • I/O Device Management
- ▶ • Security and Protection
- ▶ • Networking and Communication

Process Management

- ▶ • Handles process creation, execution, and termination
- ▶ • Schedules CPU time among processes
- ▶ • Provides inter-process communication (IPC)

Memory Management

- ▶ • Allocates and deallocates memory to processes
- ▶ • Manages virtual memory and paging
- ▶ • Ensures protection and sharing of memory

File System Management

- ▶ • Organizes and manages files on storage devices
- ▶ • Controls access permissions
- ▶ • Ensures data integrity

I/O Device Management

- ▶ • Coordinates communication between devices and system
- ▶ • Uses drivers to manage devices
- ▶ • Buffers and queues I/O operations

Security and Protection

- ▶ • Ensures authentication and authorization
- ▶ • Prevents malicious access
- ▶ • Protects resources and data

Networking Functions

- ▶ • Supports communication between computers
- ▶ • Implements network protocols
- ▶ • Manages network connections

Conclusion

- ▶ Modern operating systems balance efficiency, security, and usability.
- ▶ They manage hardware, processes, memory, I/O, files, and networks.
- ▶ Understanding OS principles is essential for computer science students.

Client-Server Distributed OS vs Single-User Operating Systems

Operating System Principles -
SKU 6.1.2

Introduction

- ▶ Operating systems can be classified based on how they manage users and resources.
- ▶ This presentation compares client-server distributed OS with single-user OS.

Client-Server Distributed Operating System

- ▶ • Resources distributed across multiple machines
- ▶ • Clients request services from server machines
- ▶ • Supports scalability, sharing, and communication
- ▶ • Fault tolerance through redundancy

Architecture of Client-Server Distributed OS

- ▶ • Client layer: User applications and interfaces
- ▶ • Server layer: File servers, database servers, computation servers
- ▶ • Communication layer: Network protocols and message-passing

Advantages of Client-Server Distributed OS

- ▶ • Efficient resource sharing
- ▶ • Centralized management
- ▶ • Improved scalability
- ▶ • High availability

Limitations of Client-Server Distributed OS

- ▶ • Higher network dependency
- ▶ • Complex design and management
- ▶ • Security concerns across systems

Single-User Operating System

- ▶ • Designed for one user at a time
- ▶ • Commonly used in personal computers
- ▶ • Simple resource management
- ▶ • Low overhead and easy to manage

Characteristics of Single-User OS

- ▶ • Provides dedicated access to CPU and memory
- ▶ • Limited multitasking capabilities
- ▶ • Focus on usability and simplicity

Comparison: Distributed OS vs Single-User OS

- ▶ • Distributed OS: Multi-system, network-based, service sharing
- ▶ • Single-User OS: Standalone system, one user, simpler management
- ▶ • Distributed OS offers scalability; single-user OS offers simplicity

Use Cases

- ▶ • Distributed OS: Cloud systems, corporate networks, web services
- ▶ • Single-User OS: Laptops, desktops, personal devices

Conclusion

- ▶ Client-server distributed OS provides scalability and resource sharing.
- ▶ Single-user OS focuses on simplicity and ease of use.
- ▶ Both serve different needs in computing environments.

The Concept of a Logical Layer

Operating System Principles -
SKU 6.1.3

Introduction

- ▶ A logical layer is an abstraction used to hide complexity in system design.
- ▶ It divides a system into manageable layers with defined responsibilities.

What is a Logical Layer?

- ▶ • A conceptual division within a system
- ▶ • Separates functions into layers for clarity
- ▶ • Each layer interacts only with adjacent layers
- ▶ • Provides abstraction and modularity

Purpose of Logical Layers

- ▶ • Reduce system complexity
- ▶ • Improve maintainability
- ▶ • Enable parallel development
- ▶ • Enhance scalability and flexibility

Logical Layers in Operating Systems

- ▶ • Hardware Layer
- ▶ • Kernel Layer
- ▶ • System Call Interface Layer
- ▶ • User Application Layer

Hardware Layer

- ▶ • Lowest layer of the system
- ▶ • Consists of CPU, memory, I/O devices
- ▶ • OS abstracts hardware complexities from users

Kernel Layer

- ▶ • Core layer of the OS
- ▶ • Responsible for resource allocation
- ▶ • Manages memory, processes, and devices

System Call Interface Layer

- ▶ • Provides interface between user programs and kernel
- ▶ • Allows programs to request OS services
- ▶ • Ensures controlled access to hardware

User Application Layer

- ▶ • Topmost layer
- ▶ • Contains user programs and interfaces
- ▶ • Interacts with OS through system calls

Benefits of Layered Design

- ▶ • Clear separation of responsibilities
- ▶ • Easier debugging and updates
- ▶ • More portable and modular OS

Conclusion

- ▶ Logical layers provide abstraction and structure within an OS.
- ▶ They simplify design, improve maintainability, and support modular systems.
- ▶ Layered architecture remains fundamental in modern operating systems.

APIs and Middleware

Operating System Principles -
SKU 6.1.4

Overview

- ▶ • What are APIs?
- ▶ • What is Middleware?
- ▶ • Role in Operating Systems
- ▶ • Examples and Use Cases

What Are APIs?

- ▶ • API = Application Programming Interface
- ▶ • Defines rules for communication between software components
- ▶ • Provides abstraction from low-level OS functionality
- ▶ • Common types: Web APIs, OS APIs, Library APIs

Role of APIs in Operating Systems

- ▶ • Acts as an interface between applications and OS
- ▶ • System call APIs allow programs to interact with hardware
- ▶ • Provides standardized access to OS resources
- ▶ • Examples: POSIX API, Windows API

What Is Middleware?

- ▶ • Software layer between OS and distributed applications
- ▶ • Facilitates communication and data management
- ▶ • Handles interoperability across systems
- ▶ • Makes distributed systems appear unified

Types of Middleware

- ▶ • Message-oriented middleware (MOM)
- ▶ • Database middleware
- ▶ • Remote Procedure Call (RPC) middleware
- ▶ • Object request brokers (e.g., CORBA)
- ▶ • Web middleware

APIs vs Middleware

- ▶ • APIs: Define how software components interact
- ▶ • Middleware: Provides runtime services enabling distributed communication
- ▶ • APIs are interfaces; Middleware is an intermediary layer
- ▶ • Both enhance OS functionality and software portability

Common Use Cases

- ▶ • APIs enabling mobile apps to access device hardware
- ▶ • Middleware linking enterprise applications
- ▶ • Web service integration via REST APIs
- ▶ • Cloud-based distributed systems

Summary

- ▶ • APIs define how software interacts with OS features
- ▶ • Middleware enables scalable, distributed communication
- ▶ • Both are essential concepts in OS principles
- ▶ • They promote abstraction, modularity, and interoperability

Resource Usage and Management

Operating System Principles -
SKU 6.1.5

Overview

- ▶ • Types of system resources
- ▶ • How application software uses resources
- ▶ • How system software manages resources
- ▶ • OS role and examples

Types of System Resources

- ▶ • CPU time
- ▶ • Memory (RAM)
- ▶ • Storage (Disk I/O)
- ▶ • Network bandwidth
- ▶ • Input/Output devices

Application Software: Resource Usage

- ▶ • Requests resources from OS
- ▶ • Executes tasks using CPU and memory allocated
- ▶ • Uses OS APIs for file access and I/O
- ▶ • Cannot directly manage hardware

Examples of Application Resource Use

- ▶ • Browsers consuming CPU for rendering pages
- ▶ • Games using GPU and memory
- ▶ • Word processors accessing disk for saving files

System Software: Resource Management

- ▶ • Controls and allocates hardware resources
- ▶ • Manages memory and processes
- ▶ • Schedules CPU tasks
- ▶ • Manages file systems and device drivers

Operating System as Resource Manager

- ▶ • Ensures fair and efficient resource allocation
- ▶ • Prevents conflicts between applications
- ▶ • Handles multitasking and process isolation
- ▶ • Provides security and access control

System Call Interface

- ▶ • Interface between user applications and OS
- ▶ • Applications request OS services (e.g., read, write, allocate memory)
- ▶ • OS validates and executes requests

Resource Protection and Control

- ▶ • Prevents unauthorized resource access
- ▶ • Implements user permissions and controls
- ▶ • Uses virtual memory for isolation

Application vs System Software

- ▶ • Application software uses resources
- ▶ • System software allocates and manages resources
- ▶ • Applications depend on OS for safe and efficient execution

Summary

- ▶ • Application software demands hardware resources
- ▶ • System software controls distribution of those resources
- ▶ • OS ensures performance, safety, and fairness
- ▶ • Understanding resource interaction is key to OS design

Kernel and User Mode in an Operating System

Operating System Principles -
SKU 6.1.6

Overview

- ▶ • What are CPU modes?
- ▶ • User mode vs Kernel mode
- ▶ • Switching modes
- ▶ • Role in OS protection and security

CPU Modes

- ▶ • Modern CPUs operate in at least two modes:
 - ▶ - User Mode (Unprivileged)
 - ▶ - Kernel Mode (Privileged)
- ▶ • Determines what instructions a program can execute
- ▶ • Ensures system stability and security

User Mode

- ▶ • Used by application programs
- ▶ • Limited access to hardware and memory
- ▶ • Cannot execute privileged instructions
- ▶ • Must request OS services using system calls

Kernel Mode

- ▶ • Full access to hardware and memory
- ▶ • Used by OS core components
- ▶ • Executes privileged instructions
- ▶ • Responsible for process control, memory management, and I/O

Why Two Modes?

- ▶ • Protects OS from faulty applications
- ▶ • Prevents direct hardware access by user apps
- ▶ • Ensures controlled resource usage
- ▶ • Provides security through isolation

System Calls: Transition Between Modes

- ▶ • Applications cannot directly access kernel
- ▶ • System calls provide controlled entry
- ▶ • CPU switches from user mode → kernel mode
- ▶ • After service, returns to user mode

Examples of Privileged Operations

- ▶ • Changing memory mapping
- ▶ • Accessing I/O devices
- ▶ • Modifying interrupt tables
- ▶ • Managing processes and threads

Security and Protection

- ▶ • Kernel mode enforces security boundaries
- ▶ • User apps run isolated from system internals
- ▶ • Prevents malicious or accidental OS corruption

Modern OS Examples

- ▶ • Windows: Ring 0 (kernel) and Ring 3 (user)
- ▶ • Linux: Kernel mode vs user mode separation
- ▶ • Android/iOS: App sandboxing enforced via user mode

Summary

- ▶ • Two modes ensure safe and efficient system operation
- ▶ • User mode runs applications with restrictions
- ▶ • Kernel mode provides full control for OS tasks
- ▶ • System calls are the bridge between modes

Interrupt Processing

Operating System Principles -
SKU 6.1.7

Overview

- ▶ • What is an interrupt?
- ▶ • Types of interrupts
- ▶ • Interrupt handling steps
- ▶ • Role of the OS and hardware

What Is an Interrupt?

- ▶ • A signal that temporarily halts CPU execution
- ▶ • CPU stops current task and handles the event
- ▶ • Restores state and resumes execution
- ▶ • Enables multitasking and responsiveness

Types of Interrupts

- ▶ • Hardware interrupts
 - ▶ - Generated by devices (keyboard, disk, network)
- ▶ • Software interrupts
 - ▶ - Generated by programs (system calls, traps)
- ▶ • Timer interrupts
 - ▶ - Used by OS for scheduling tasks

Interrupt Handling Process

- ▶ 1. Interrupt occurs
- ▶ 2. CPU finishes current instruction
- ▶ 3. CPU switches to kernel mode
- ▶ 4. Interrupt vector identifies handler
- ▶ 5. Interrupt Service Routine (ISR) runs
- ▶ 6. CPU restores state and resumes execution

Interrupt Vector Table

- ▶ • Table storing addresses of interrupt handlers
- ▶ • Each interrupt type has an entry
- ▶ • Ensures quick lookup by CPU
- ▶ • Critical for real-time response

Interrupt Service Routine (ISR)

- ▶ • Function executed when interrupt occurs
- ▶ • Performs minimal urgent tasks
- ▶ • Should be fast to avoid delays
- ▶ • May defer work to background processes

Interrupt Masking and Priorities

- ▶ • CPU may disable (mask) certain interrupts
- ▶ • High-priority interrupts preempt low-priority ones
- ▶ • Used to prevent conflicts and ensure responsiveness

OS Role in Interrupt Processing

- ▶ • Manages interrupt handlers
- ▶ • Saves and restores process state
- ▶ • Ensures safe access to shared resources
- ▶ • Enables multitasking via timer interrupts

Examples of Interrupts

- ▶ • Keyboard input interrupt
- ▶ • Hard disk read completion
- ▶ • Network packet arrival
- ▶ • Process time slice expiration

Summary

- ▶ • Interrupts improve system efficiency and responsiveness
- ▶ • CPU reacts to events via ISRs
- ▶ • OS controls interrupt processing and scheduling
- ▶ • Essential for modern multitasking systems

Device List and Driver I/O Queue

Operating System Principles -
SKU 6.1.8

Overview

- ▶ • What is a device list?
- ▶ • What are device drivers?
- ▶ • I/O queue structure
- ▶ • OS role in device and I/O management

Device List

- ▶ • Data structure maintained by the OS
- ▶ • Contains information about all connected devices
- ▶ • Includes device type, ID, status, and driver mapping
- ▶ • Enables OS to track and manage hardware

Device Drivers

- ▶ • Software modules that control hardware devices
- ▶ • Translate OS commands into device-specific actions
- ▶ • Provide an abstraction layer
- ▶ • Essential for hardware compatibility

OS Interaction with Drivers

- ▶ • OS sends I/O requests through system calls
- ▶ • Driver interprets and processes the request
- ▶ • Driver communicates with hardware
- ▶ • Driver updates device status in device list

I/O Queue

- ▶ • Queue storing pending I/O requests for a device
- ▶ • Maintained by the device driver
- ▶ • Ensures orderly processing of requests
- ▶ • Prevents overload and supports scheduling

I/O Queue Structure

- ▶ • Typically FIFO, but may use optimized scheduling
- ▶ • Contains:
 - ▶ - Request type (read/write)
 - ▶ - Memory buffer location
 - ▶ - Process ID
 - ▶ - Device command parameters

I/O Scheduling

- ▶ • OS or driver decides request order
- ▶ • Common algorithms:
 - ▶ - FIFO
 - ▶ - SSTF (Shortest Seek Time First)
 - ▶ - SCAN/LOOK
- ▶ • Improves performance and reduces latency

Example: I/O Request Flow

- ▶ 1. Application makes I/O request
- ▶ 2. OS validates request
- ▶ 3. OS sends request to the driver
- ▶ 4. Driver adds request to I/O queue
- ▶ 5. Device processes request
- ▶ 6. Driver updates OS, OS resumes application

Importance

- ▶ • Central to device management
- ▶ • Ensures efficient and safe hardware access
- ▶ • Prevents conflicts between processes
- ▶ • Foundation for multitasking and concurrency

Summary

- ▶ • Device list tracks hardware devices
- ▶ • Drivers translate OS commands into hardware actions
- ▶ • I/O queues manage pending requests
- ▶ • OS and drivers cooperate to ensure smooth I/O operations